

Configurability Validation Gallery

Configurability Validation Gallery

This gallery documents the live reproduction of the three configurability dimensions reported in Section~5.4 of the thesis. Each screenshot was captured against a clean checkout of the source repository, brought up through the `docker-compose.postgres.yml` Compose file, with no application source code modified between dimensions. Screenshots are grouped by dimension.

UI evidence (editor and admin views) is paired with infrastructure evidence (terminal output from the running containers and the PostgreSQL database), so each claim can be verified at both layers.

A. Setup

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
datacurationtool-legal-mock-1	datacurationtool-legal-mock	"docker-entrypoint.s..."	legal-mock	2 minutes ago	Up 2 minutes	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
datacurationtool-medical-mock-1	datacurationtool-medical-mock	"docker-entrypoint.s..."	medical-mock	2 minutes ago	Up 2 minutes	0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
datacurationtool-memorise-api-1	datacurationtool-memorise-api	"docker-entrypoint.s..."	memorise-api	2 minutes ago	Up 2 minutes	0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
datacurationtool-memorise-ui-1	datacurationtool-memorise-ui	"docker-entrypoint.s..."	memorise-ui	2 minutes ago	Up 2 minutes	0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
datacurationtool-postgres-1	postgres:16-alpine	"docker-entrypoint.s..."	postgres	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Figure 1: `docker compose -f docker-compose.postgres.yml ps` after starting the reproduction stack: all five services (postgres, memorise-api, memorise-ui, legal-mock, medical-mock) come up under a single Compose file, with Postgres reporting (**healthy**). The entire configurability validation runs from this one declarative artefact.

B. Dimension 1. Drop-In URL Replacement

SERVICES						
API Endpoints		Server (http://localhost:3001)				
				EDIT REFRESH		
Endpoint	URL	Adapter	Status	Latency	Last Checked	
Named Entity Recognition	https://ner-api.dev.memorise.sdu.dk/recogn...	sdu-ner	Up	256ms	10:05:14 PM	
Text Segmentation	https://textseg-api.dev.memorise.sdu.dk/seg...	sdu-segment	Up	183ms	10:05:14 PM	
Semantic Classification	https://semtag-api.dev.memorise.sdu.dk/clas...	sdu-classify	Up	258ms	10:05:14 PM	
Machine Translation	https://mt-api.dev.memorise.sdu.dk/translate	sdu-translate	Up	362ms	10:05:14 PM	
Supported Languages	https://mt-api.dev.memorise.sdu.dk/support...	default	Up	361ms	10:05:14 PM	

Figure 2: Admin Services panel before any reconfiguration. The four NLP endpoints (NER, Segmentation, Classification, Translation) point at the SDU upstream services (`*.dev.memorise.sdu.dk/...`) with the home adapters (`sdu-ner`, `sdu-segment`, `sdu-classify`, `sdu-translate`). All endpoints report **Up**. This is the Holocaust-testimony deployment baseline.

API Endpoints Server (http://localhost:3001) EDIT REFRESH					
Endpoint	URL	Adapter	Status	Latency	Last Checked
Semantic Classification	http://legal-mock:5001/classify	sdu-classify	Up	11ms	1:39:58 PM
Named Entity Recognition	http://legal-mock:5001/recognize	sdu-ner	Up	11ms	1:39:58 PM
Text Segmentation	http://legal-mock:5001/segment	sdu-segment	Up	11ms	1:39:58 PM
Machine Translation	http://legal-mock:5001/translate	sdu-translate	Up	11ms	1:39:58 PM
Supported Languages	http://legal-mock:5001/supported_languages	default	Up	11ms	1:39:58 PM

Figure 3: Same panel after a `PUT /api/config` issued from the admin UI. All four endpoints now point at `http://legal-mock:5001/...`; the adapter selection is *unchanged* (`sdu-ner` etc.). The mock mirrors the SDU upstream contract on a per-path and per-payload basis, so the home adapters consume its responses verbatim. A URL flip alone is therefore sufficient to retarget the platform onto a same-shape upstream.

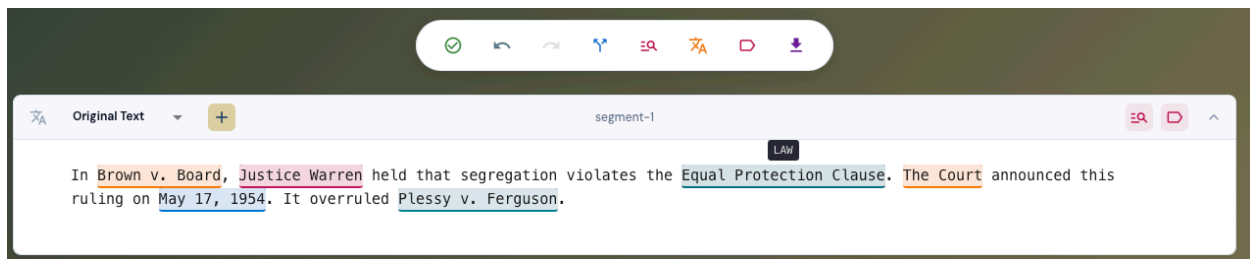


Figure 4: Editor displaying the *Brown v. Board* excerpt with the six entities returned by the legal mock highlighted: *Brown v. Board* (ORG), *Justice Warren* (PERSON), *Equal Protection Clause* (LAW), *The Court* (ORG), *May 17, 1954* (DATE), *Plessy v. Ferguson* (LAW). The LAW tooltip is visible above *Equal Protection Clause*. The unmodified UI renders legal-domain NER results end-to-end after the URL flip.

```

legal-mock-1 | [legal-mock] loaded 3 sample document(s)
legal-mock-1 | [legal-mock] listening on http://localhost:5001
legal-mock-1 | [legal-mock] 2026-05-05T11:39:58.822Z HEAD /classify
legal-mock-1 | [legal-mock] 2026-05-05T11:39:58.825Z HEAD /recognize
legal-mock-1 | [legal-mock] 2026-05-05T11:39:58.825Z HEAD /segment
legal-mock-1 | [legal-mock] 2026-05-05T11:39:58.826Z HEAD /supported_languages
legal-mock-1 | [legal-mock] 2026-05-05T11:39:58.826Z HEAD /translate
legal-mock-1 | [legal-mock] 2026-05-05T11:40:28.884Z POST /recognize
legal-mock-1 | [legal-mock] 2026-05-05T11:42:00.943Z GET /supported_languages
legal-mock-1 | [legal-mock] 2026-05-05T11:42:43.490Z POST /recognize

```

Figure 5: `docker compose ... logs legal-mock | tail -20` showing the inbound POST `/recognize` requests issued by `memorise-api` when the editor triggered NER, alongside the HEAD health probes from the admin panel and the GET `/supported_languages` request from the translation tab. The requests reached the SDU-shape mock at the SDU endpoint paths exactly as expected.

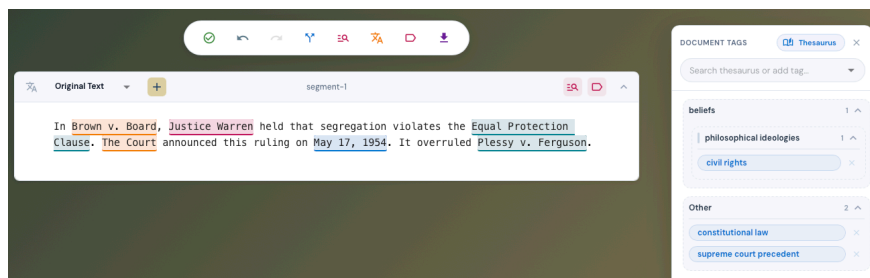
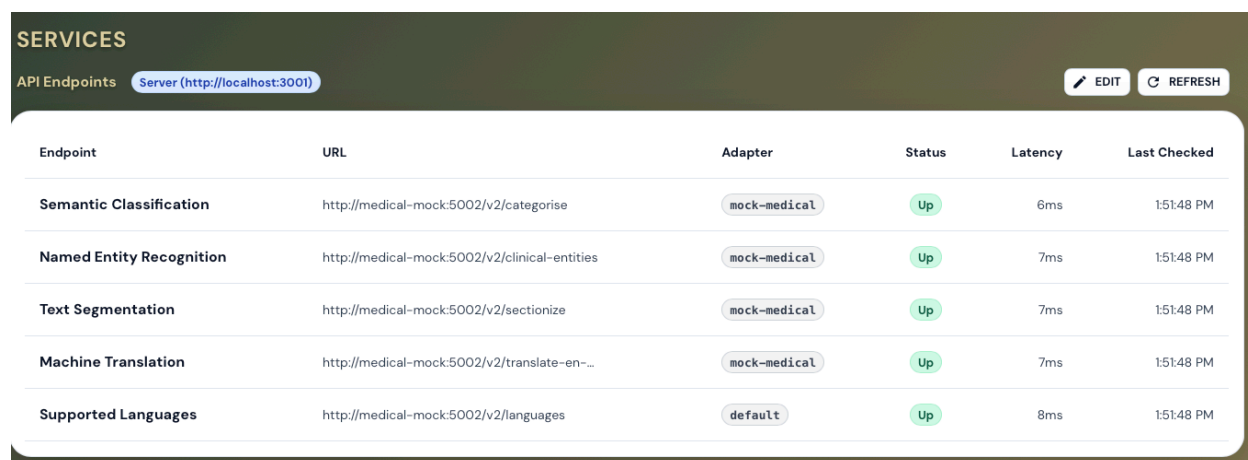


Figure 6: Document Tags panel listing the three legal classification labels returned by `legal-mock /classify` for the same workspace: `constitutional law`, `civil rights`, `supreme court precedent`. The UI groups them under its existing thesaurus hierarchy (`beliefs > philosophical ideologies`); platform-side post-processing behaves identically regardless of the upstream service. Same demonstration as the NER captures above, applied to the `classify` endpoint.

C. Dimension 2. Shape-Translating Adapter



Endpoint	URL	Adapter	Status	Latency	Last Checked
Semantic Classification	http://medical-mock:5002/v2/categorise	mock-medical	Up	6ms	1:51:48 PM
Named Entity Recognition	http://medical-mock:5002/v2/clinical-entities	mock-medical	Up	7ms	1:51:48 PM
Text Segmentation	http://medical-mock:5002/v2/sectionize	mock-medical	Up	7ms	1:51:48 PM
Machine Translation	http://medical-mock:5002/v2/translate-en-...	mock-medical	Up	7ms	1:51:48 PM
Supported Languages	http://medical-mock:5002/v2/languages	default	Up	8ms	1:51:48 PM

Figure 7: Admin Services panel after a second PUT `/api/config`. The four NLP endpoints now target `http://medical-mock:5002/v2/...` paths (`clinical-entities`, `sectionize`, `categorise`, `translate-en-medical`) with adapter selection switched to `mock-medical`. All Up at sub-10 ms latency. The AdapterRegistry advertises both home and medical adapters; the swap is purely configuration, no rebuild.

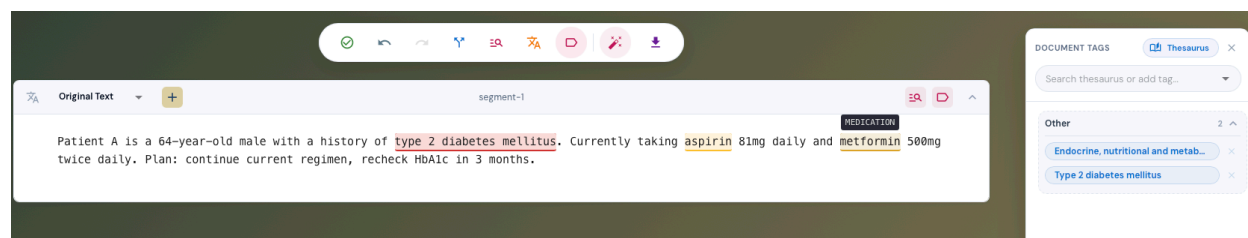


Figure 8: Editor with the synthetic SOAP note rendered after NER and Classify ran against the medical mock. The three medical entities (`type 2 diabetes mellitus DISEASE`, `aspirin MEDICATION`, `metformin MEDICATION`) appear as inline highlights, with the `MEDICATION` tooltip visible. The Document Tags panel on the right shows the ICD-10 chapter `Endocrine, nutritional and metabolic diseases` and the subcategory `Type 2 diabetes mellitus` returned by `medical-mock /v2/categorise`. The medical adapter normalises a divergent upstream shape into the platform’s internal value objects, end-to-end, without any UI change.

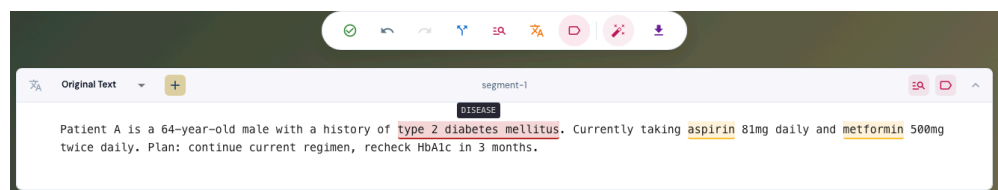


Figure 9: Same workspace at higher zoom; the `DISEASE` tooltip above `type 2 diabetes mellitus` is fully legible.

```

medical-mock-1 | [medical-mock] loaded 3 sample document(s)
medical-mock-1 | [medical-mock] listening on http://localhost:5002
medical-mock-1 | [medical-mock] 2026-05-05T11:51:48.993Z HEAD /v2/categorise
medical-mock-1 | [medical-mock] 2026-05-05T11:51:48.996Z HEAD /v2/clinical-entities
medical-mock-1 | [medical-mock] 2026-05-05T11:51:48.997Z HEAD /v2/sectionize
medical-mock-1 | [medical-mock] 2026-05-05T11:51:48.997Z HEAD /v2/translate-en-medical
medical-mock-1 | [medical-mock] 2026-05-05T11:51:48.998Z HEAD /v2/languages
medical-mock-1 | [medical-mock] 2026-05-05T11:52:58.202Z GET /v2/languages
medical-mock-1 | [medical-mock] 2026-05-05T11:53:20.462Z POST /v2/clinical-entities
medical-mock-1 | [medical-mock] 2026-05-05T11:53:33.394Z POST /v2/categorise

```

Figure 10: `docker compose ... logs medical-mock | tail -20`. The inbound requests are `POST /v2/clinical-entities` and `POST /v2/categorise` — fundamentally different paths from the legal mock’s `POST /recognize` and `/classify`. The platform spoke to the upstream over the new contract; the adapter bridged the gap.

```

{
  "metadata": {
    "model": "mock-bioBERT-v1",
    "version": "2026.04",
    "processing_time_ms": 42
  },
  "entities": {
    "DISEASE": [
      {
        "span": [
          50,
          74
        ],
        "text": "type 2 diabetes mellitus",
        "confidence": 0.94,
        "codes": [
          {
            "system": "ICD10",
            "code": "E11"
          }
        ]
      }
    ],
    "MEDICATION": [
      {
        "span": [
          93,
          100
        ],
        "text": "aspirin",
        "confidence": 0.97,
        "codes": [
          {
            "system": "RxNorm",
            "code": "1191"
          }
        ],
        "attributes": {
          "dose": "81mg",
          "frequency": "daily",
          "route": "PO"
        }
      }
    ],
    "ANATOMY": [],
    "PROCEDURE": []
  }
}

```

Figure 11: Raw response from `medical-mock /v2/clinical-entities` printed via `curl ... | jq`, split across two columns for legibility. The payload carries a `metadata` block, entity buckets keyed by `DISEASE` and `MEDICATION`, ICD-10 and RxNorm codes, structured `attributes` (dose, frequency, route), and per-entity confidence. The `MockMedicalNerAdapter` collapses this shape into the flat `{start, end, entity, score}` form consumed by the application layer (the upstream `type` field is renamed to `entity` during normalization).

D. Dimension 3. Storage Adapter Swap

key	url	adapter
ner	http://medical-mock:5002/v2/clinical-entities	mock-medical
segment	http://medical-mock:5002/v2/sectionize	mock-medical
classify	http://medical-mock:5002/v2/categorise	mock-medical
translate	http://medical-mock:5002/v2/translate-en-medical	mock-medical
translate-languages	http://medical-mock:5002/v2/languages	

Figure 12: `psql -c "SELECT key, url, adapter FROM endpoint_config;"` issued inside the running `postgres` container. Five rows: the four NLP endpoints (with `medical-mock` URLs and adapter `mock-medical`) plus the `translate-languages` helper. The endpoint configuration mutated by Dimensions 1 and 2 was persisted into a relational table by the `PostgresAdapter`, not into a JSON file — queryable through standard SQL tooling, with no application code aware of the swap.

id	name	owner_id
1f514afd-6759-4d27-a6c4-a06eb49c71aa	Workspace A	d3abec7d-09b9-4901-ba4e-24254eedb6a7
1fece8fa-c7bc-4013-b90b-b26628437b54	Workspace B	d3abec7d-09b9-4901-ba4e-24254eedb6a7
aab0b6e9-0274-478d-9407-137ad9efaaa8	Workspace C	d3abec7d-09b9-4901-ba4e-24254eedb6a7
924a3132-aa6f-4d53-9925-b7a0a6e8d57e	Brown	d3abec7d-09b9-4901-ba4e-24254eedb6a7
cb71ea8f-85f3-4e28-b465-66a392bfb354	medical	d3abec7d-09b9-4901-ba4e-24254eedb6a7

Figure 13: `SELECT id, name, owner_id FROM workspaces;` showing five rows: the three platform-seeded workspaces (Workspace A, Workspace B, Workspace C, created by `WorkspaceApplicationService.seedForOwner` on first admin login) plus the two workspaces created during this reproduction (Brown, medical). All share the admin’s UUID as `owner_id`. Workspace state, not just configuration, is materialised in Postgres.

```

adamturcan [DataCurationTool] [Pmain # 710 | ~14] [143ms] docker compose -f docker-compose.postgres.yml exec postgres psql -U memorise -d memorise -c "SELECT key, url FROM
endpoint_config;"
key | url
ner | http://medical-mock:5002/v2/clinical-entities
segment | http://medical-mock:5002/v2/sectionize
classify | http://medical-mock:5002/v2/categorise
translate | http://medical-mock:5002/v2/translate-en-medical
translate-languages | http://medical-mock:5002/v2/languages
(5 rows)

adamturcan [DataCurationTool] [Pmain # 710 | ~14] [1284ms] docker compose -f docker-compose.postgres.yml down
[+] Running 6/6
  Container datacurationtool-memorise-ui-1 Removed
  Container datacurationtool-memorise-api-1 Removed
  Container datacurationtool-legal-mock-1 Removed
  Container datacurationtool-medical-mock-1 Removed
  Container datacurationtool-postgres-1 Removed
  Network datacurationtool_default Removed

adamturcan [DataCurationTool] [Pmain # 710 | ~14] [20.512s] docker volume ls | grep postgres
local datacurationtool_postgres-data

adamturcan [DataCurationTool] [Pmain # 710 | ~14] [136ms] docker compose -f docker-compose.postgres.yml up -d
[+] Running 6/6
  Network datacurationtool_default Created
  Container datacurationtool-medical-mock-1 Started
  Container datacurationtool-postgres-1 Healthy
  Container datacurationtool-legal-mock-1 Started
  Container datacurationtool-memorise-api-1 Started
  Container datacurationtool-memorise-ui-1 Started

adamturcan [DataCurationTool] [Pmain # 710 | ~14] [5.953s] sleep 10
adamturcan [DataCurationTool] [Pmain # 710 | ~14] [10.022s] docker compose -f docker-compose.postgres.yml exec postgres psql -U memorise -d memorise -c "SELECT key, url FROM
endpoint_config;"
key | url
ner | http://medical-mock:5002/v2/clinical-entities
segment | http://medical-mock:5002/v2/sectionize
classify | http://medical-mock:5002/v2/categorise
translate | http://medical-mock:5002/v2/translate-en-medical
translate-languages | http://medical-mock:5002/v2/languages
(5 rows)

```

Figure 14: Restart survival sequence captured top-to-bottom: (1) `psql` returns the medical-mock endpoint config (5 rows), (2) `docker compose ... down` removes the containers without `-v` so the volume is preserved, (3) `docker volume ls | grep postgres` shows `datacurationtool_postgres-data` still present, (4) `docker compose ... up -d` re-creates the containers, (5) `psql` after the restart returns identical 5 rows. The platform’s persistent state is external, multi-process-safe, and survives container teardown without any application-side handling — the property §5.4.4 claims.